Esercitazione Python n. 7 -- 17 Novembre 2025

Per risolvere gli esercizi **scrivete la soluzione di ogni esercizio nel file .py relativo**, che trovate nella cartella dell'esercitazione (ad esempio, per l'esercizio 1 scrivete il vostro programma nel file Ex1.py, per l'esercizio 2, nel file Ex2.py, e così via). Notate che ogni file incorpora del codice python per eseguire alcuni test sulla funzione. NON modificate questo codice, ma **SCRIVETE SOLO il contenuto della funzione**. Per farlo usare l'ambiente Spyder.

Potete riconsegnare l'esercitazione attraverso il sito classroom. L'esercitazione non verrà comunque corretta singolarmente.

In ogni esercizio, se non diversamente richiesto, potete sempre assumere che gli input forniti siano coerenti con la traccia (ad esempio, se l'esercizio chiede di dare in input alla funzione un intero positivo, potete assumere che l'input sia sempre un numero intero maggiore di zero, e non è necessario nel codice effettuare controlli per gestire casi diversi da questo).

Esercizi

- **Ex1(l)** Completare la funzione Python Ex1(l) (contenuta nel file Ex1.py) realizzando una funzione che, prendendo in ingresso una lista **l** di stringhe, restituisce una lista che contiene le lunghezze delle stringhe contenute in **l**. Nella lista restituita le lunghezze devono avere lo stesso ordine delle stringhe a cui appartengono. Ad esempio, se **l**=`['tanto','va','la','gatta','al','lardo']` la funzione deve restituire `[5,2,2,5,2,5]`.

- **Ex2(l)** Completare la funzione Python Ex2(l) (contenuta nel file Ex2.py) realizzando una funzione che, prendendo in ingresso una lista **l** di stringhe, restituisce la lunghezza massima di una stringa contenuta in **l**. Ad esempio, se **l**=`['tanto','va','la','gatta','al','lardo']` la funzione deve restituire `5`.

- **Ex3(l1, l2)** Completare la funzione Python Ex3(l1,l2) (contenuta nel file Ex3.py), realizzando una funzione che, ricevendo in ingresso due liste **l1** e **l2** contenenti numeri interi, e tali che `len(l2)>= len(l1)`, restituisca una lista composta dalla somma degli interi che si trovano nella stessa posizione in **l1** e **l2**. Se **l1** è più corta si assuma che gli interi mancanti siano uguali a `0`. Ad esempio, se **l1** vale `[3,6]` e **l2** vale `[3,4,9]` la funzione deve restituire `[6, 10, 9]`. Se **l1** ed **l2** sono entrambe vuote (cioè pari a `[]`), la funzione deve restituire una lista vuota.

- **Ex4 (start, n)** Completare la funzione Python Ex4(start,n) (contenuta nel file Ex4.py), realizzando una funzione che riceve in ingresso un intero **start** ed un intero non negativo **n** e restituisce la lista dei primi n numeri >= **start**. Se **n** vale zero, la funzione deve restituire una lista vuota. Ad esempio, se **start** vale `4` e **n** vale `3` la funzione deve restituire `[4,5,6]`.

- **Ex5(l,n)** Completare la funzione Python Ex5(l,n) (contenuta nel file Ex5.py), realizzando una funzione che prende in ingresso una lista **l** di stringhe e un intero non negativo **n** e restituisce la stringa composta, in ordine, da tutti i caratteri con indice **n** di tutte le stringhe della lista. Se in una stringa in **l** non c'è il carattere di indice **n** (perché la stringa è troppo corta), la funzione deve inserire il carattere `'!'`. Se la lista in ingresso è vuota, la funzione deve restituire la stringa vuota. Ad esempio, se **l** vale `['tanto','va','la','gatta','al','lardo']` e **n** vale `3` la funzione deve restituire la stringa `'t!!t!d'`.
  Suggerimento: Usate un ciclo for sugli elementi della lista e non sugli indici. Ovviamente è possibile usare anche un ciclo for sugli indici, o un ciclo while. In questo modo però, per accedere al carattere in posizione n (se questo esiste) della i-esima stringa della lista, dovete usare due volte l'operatore di

indicizzazione, scrivendo, ad esempio, `l[i][n]`, cosa che potete evitare se il ciclo for è sugli elementi della lista.

- **Ex6(l)** Completare la funzione Python Ex6(l) (contenuta nel file Ex6.py), realizzando una funzione che prende in ingresso una lista **l** di interi e restituisce la stringa composta da soli caratteri `'P'` e `'D'` che indicano se l'intero nella lista è pari o dispari. Più precisamente, se il numero in posizione *i* in **l** è pari, la stringa restituita dalla funzione contiene `'P'` in posizione *i,* `'D'` altrimenti. Se la lista è vuota la funzione deve restituire la stringa vuota. Ad esempio, se **l** vale `[3,7,8,9]` la funzione deve restituire la stringa `'DDPD'`.

- **Ex7(l)** Completare la funzione Python Ex7(l) (contenuta nel file Ex7.py), realizzando una funzione che prende in ingresso una lista **l** di stringhe e restituisce una lista di interi, tale che ciascun intero *x* nella lista restituita è ottenuto dalla somma dei codici Unicode dei caratteri della stringa che in **l** occupa la stessa posizione che occupa *x* nella lista restituita. Ad esempio, se **l** è `['ama','ma','amaca']` la funzione dovrebbe restituire la lista `[303,206,499]`, in quanto il codice Unicode di `'a'` è `97`, quello di `'m'` è `109` e quello di `'c'` è `99` (e quindi ad esempio ad `'ama'` corrisponde l'intero `303`). Se la lista in input è vuota, la funzione deve restituire una lista vuota. *Suggerimento:* per risolvere questo esercizio c'è bisogno di un ciclo annidato (il ciclo esterno scandisce la lista **l**, quello interno scandisce la stringa individuata dal ciclo esterno in una sua iterazione).

- **Ex8(l1,l2)** Completare la funzione Python Ex8(l1,l2) (contenuta nel file Ex8.py), realizzando una funzione che prende in ingresso due liste **l1** e **l2** contenenti numeri interi e restituisce una lista contenente gli elementi di **l1** che NON compaiono in **l2.** Nella lista restituita gli elementi devono comparire in **ordine crescente**. Si noti che eventuali ripetizioni di un intero *x* in **l1** sono tutte non presenti nel risultato se *x* compare anche solo una volta in **l2**, mentre sono tutte presenti nel risultato se *x* non compare in **l2**. Ovviamente, se la lista **l1** è vuota la funzione deve restituire la lista vuota, invece se **l2** è vuota, la funzione deve restituire una lista uguale a **l1**. Ad esempio, se **l1** = `[1,3,7,2,1,-5,7]` e **l2** = `[1,3]`, la funzione deve restituire la lista `[-5,2,7,7]`.

- **Ex9(s)** Completare la funzione Python Ex9(s) (contenuta nel file Ex9.py), realizzando una funzione che prende in ingresso una stringa **s** e restituisce una lista composta da tutti e soli i caratteri alfabetici maiuscoli contenuti in **s**, senza ripetizioni. Nella lista restituita gli elementi devono comparire in **ordine crescente**. Se la stringa in ingresso è vuota, la funzione deve restituire una lista vuota. Ad esempio, se **s**=`'cIAo MAmMa'`, la funzione deve restituire la lista `['A','I','M']`.

- **Ex10(s1,s2)** Completare la funzione Python Ex10(s1,s2) (contenuta nel file Ex10.py), realizzando una funzione che prende in ingresso due stringhe **s1** e **s2** e restituisce il prefisso più lungo comune alle due stringhe. Ad esempio, se **s1** è `'amaca'` e **s2** è `'amaranto'`, la funzione deve restituire la stringa `'ama'`. Se invece, **s1** è `'amaca'` e **s2** è `'stringa'` la funzione deve restituire la stringa vuota `''`. Se (almeno) una delle due stringhe è vuota la funzione deve restituire la stringa vuota `''`.